

Discussion Module:

DSE Search Prerequisites

Review the briefest amount of material related to DSE and DSE Core, so that we may start to understand the Why and What of DSE Search.

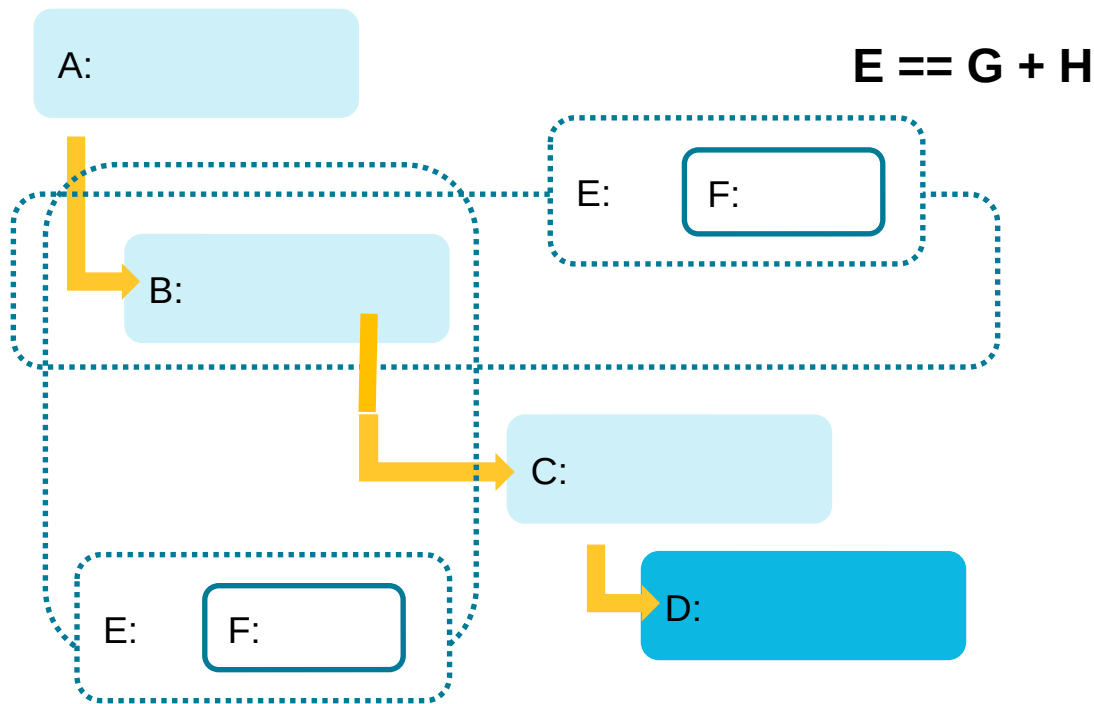
- (One) DSE Object Hierarchy
- Brewer's CAP Theorem, Consistency Level (CL)
- Four Primary functional Areas to DSE
- Index (types/options) in DSE Core

DSE Object Hierarchy

Discussion Lab:

Refrigerator Magnets - Match the terms on the right with the boxes on the left

Discussion Lab: (One) DSE Object Hierarchy



- Replication Strategy
- Keyspace
- Clustering Column
- Node (host)
- Table
- Cluster
- Primary Key
- Replication Factor
- Rack
- Data center
- Partitioning Key

And answer the questions on the Notes Page-

End of Discussion Lab:

Wikipedia.com: Brewer's CAP Theorem

WIKIPEDIA
The Free Encyclopedia



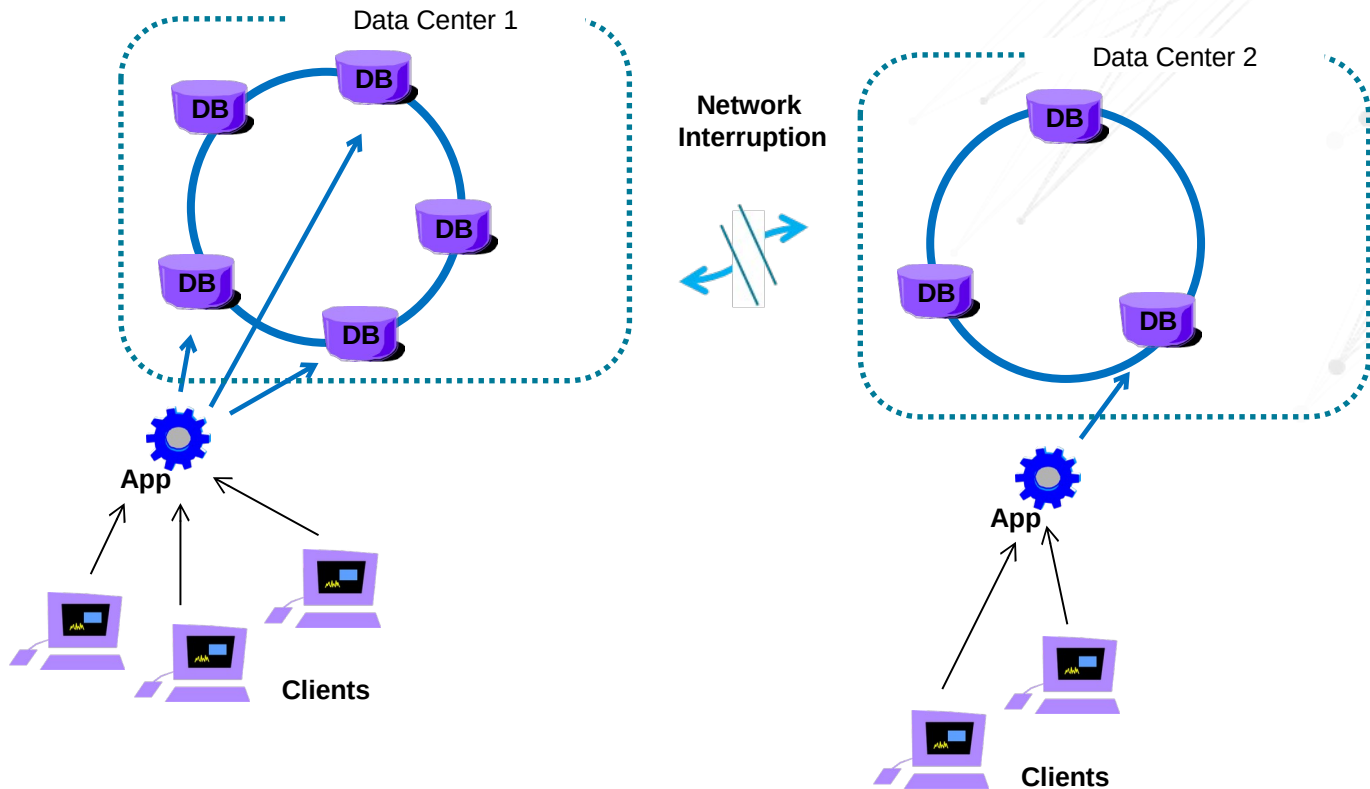
Generally, choose two:

- “Consistency - Every read receives the most recent write or an error
 - “Availability - Every request receives a (non-error) response – without guarantee that it contains the most recent write
 - “Partition - The system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes
-
- *“CAP is frequently misunderstood as if one has to choose to abandon one of the three guarantees at all times. In fact, the choice is really between consistency and availability only when a network partition or failure happens; at all other times, no trade-off has to be made.*

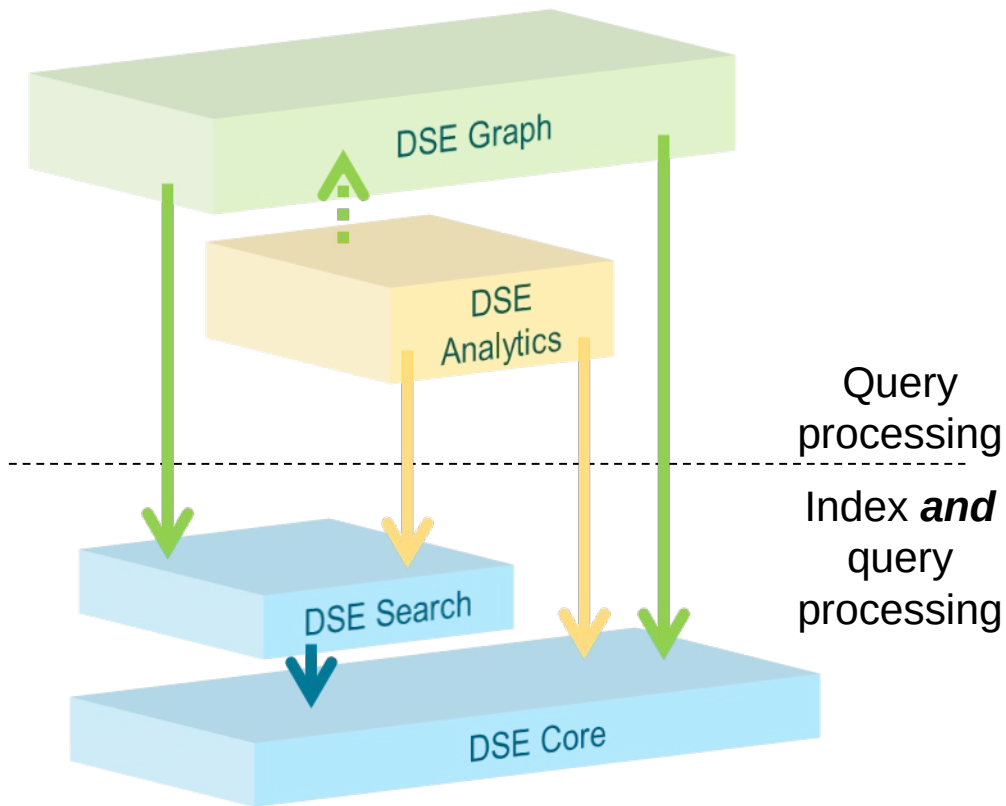
Source: https://en.wikipedia.org/wiki/CAP_theorem

000-DTSE-Search-7442-60-DM-5

CAP: Data Centers and Consistency Level (CL)

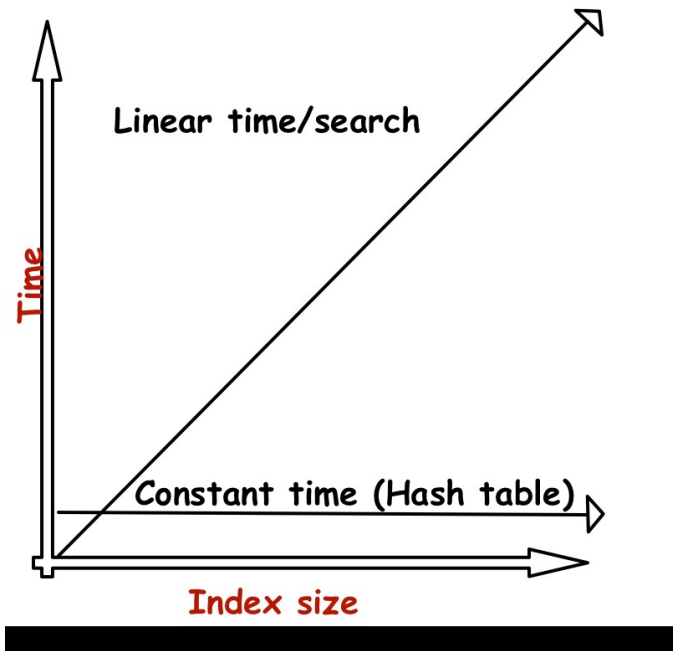
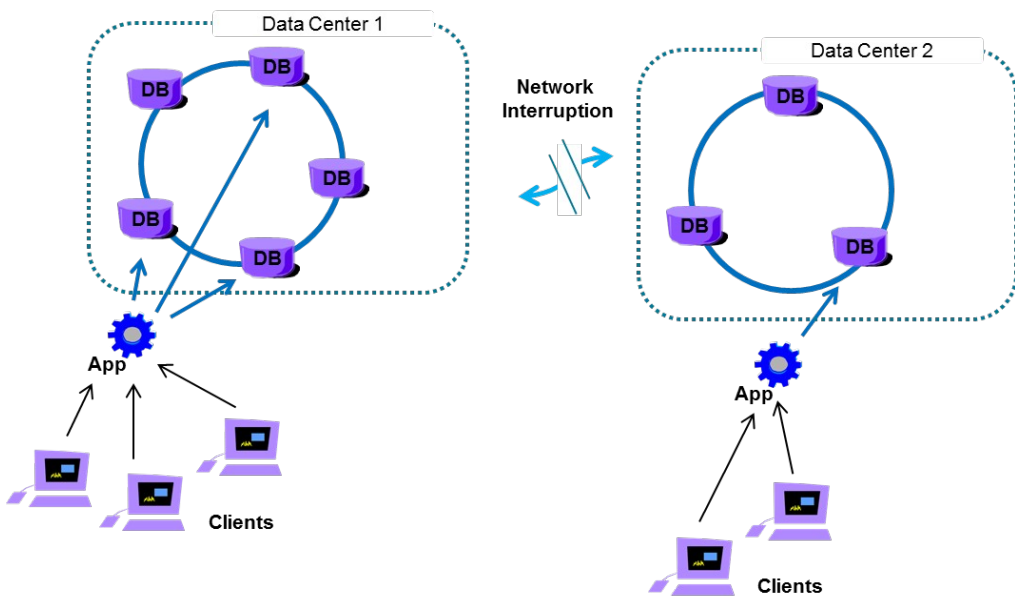


The 4 Primary Functional Areas to DSE



- Only 2 of the 4 primary functional areas to DSE provide *indexing technology*
 - DSE Core, *primarily* hash
 - DSE Search, Tf-Idf (bitmap)
- All 4 primary functional areas provide query processing
 - Differentiated query predicates, or functionality
 - Specific performance abilities

Why DSE Core: 2 of (M) Value Propositions



Wikipedia.com: B-Tree+ Index

- “In computer science, a B-tree is a self-balancing tree data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in logarithmic time.
- “An algorithm is said to take logarithmic time if $T(n) = O(\log n)$ An $O(\log n)$ algorithm is considered highly efficient, as the ratio of the number of operations to the size of the input decreases, and **tends to zero when n increases**.

WIKIPEDIA
The Free Encyclopedia



Source: <https://en.wikipedia.org/wiki/B-tree>

Source:
https://en.wikipedia.org/wiki/Time_complexity#Logarithmic_time

Wikipedia.com: (Hash Index, time constant)

WIKIPEDIA
The Free Encyclopedia



“In a well-dimensioned hash table, **the average cost (number of instructions) for each lookup is independent of the number of elements stored in the table.**

Many hash table designs also allow arbitrary insertions and deletions of key-value pairs, at (amortized[2]) **constant average cost per operation.**

Source: https://en.wikipedia.org/wiki/Hash_table

000-DTSE-Search-7442-60-DM-10

Discussion Lab:

Design

DSE Core/(hash)

- Which Query Predicates are Supported ,
- Which Queries are Core/Search?

Discussion Lab: Just Core, Which Query Predicates are Supported

CREATE TABLE t1 ...

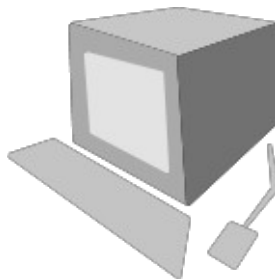
**(
col1 TEXT,
col2 TEXT, ...**

PRIMARY KEY ((col1, col2)

col3, col4)

) WITH CLUSTERING ORDER BY

(col3 DESC, col4 ASC);



SELECT * WHERE ...

col? =

col? >

AND ... / OR ...

ORDER BY

col? ;

col?, col? ;

GROUP BY

col?, col? ;



Discussion Lab: Which are Core/Search Queries

SELECT * FROM t1

WHERE col1 = 10;

WHERE col1 LIKE 'Chuck';

WHERE col1 = 10 AND col2 = 20;

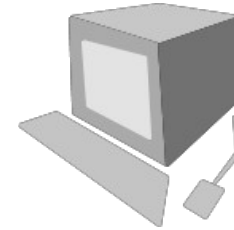
WHERE col1 = 10 AND col2 = 20 and col3 > 30;

WHERE col1 = 10 and col2 = 20 ORDER BY col3;

WHERE solr_query =

{ "q" : "(name:D* OR name:DAVE~1^8)

AND gender:M AND century:2000" }';



CREATE TABLE t1 ... ?? ;



End of Discussion Lab:

Other DSE Core (Indexes)

- **Materialized Views**
- **Secondary Indexes**
- **(Custom Indexes)**
- **SASI Indexes**

- **(DSE Query Tables)**



Copyright: Village Roadshow Pictures

DSE Core: Materialized Views

CREATE MATERIALIZED VIEW t1_v1

AS

SELECT col1, ... , col8

FROM t1

WHERE

col1 IS NOT NULL AND col2 IS NOT NULL

AND col7 IS NOT NULL

PRIMARY KEY (col1, col2, col7);

OR, PRIMARY KEY ((col8), col2, col1); OR, OR ..



CREATE TABLE t1 ...

PRIMARY KEY

((col1, col2)));

Materialized Views: You may not

```
CREATE TABLE t1
...
PRIMARY KEY
((col1, col2)) );
```

```
CREATE MATERIALIZED VIEW t1_v2 AS
SELECT col1, ... , col8
FROM t1
WHERE
    col1 IS NOT NULL AND col2 IS NOT NULL AND
    col7 IS NOT NULL AND col5 IS NOT NULL
PRIMARY KEY (col1, col2, col7, col5);
// InvalidRequest: Error from server: code=2200 [Invalid query]
// message="Cannot include more than one non-primary key column
// 'col5' in materialized view primary key"
```



Materialized Views: You may not

```
CREATE TABLE t1
..
PRIMARY KEY
((col1, col2)) );
```

```
CREATE MATERIALIZED VIEW t1_v2 AS
SELECT col1, ... , col8
FROM t1
WHERE
    col1 IS NOT NULL AND col2 IS NOT NULL AND
    col7 IS NOT NULL
PRIMARY KEY (col1, col7);
```

```
// InvalidRequest: Error from server: code=2200 [Invalid query]
// message="Cannot create Materialized View t1_v2 without
// primary key columns from base t1 (col2)"
```



Materialized Views: You may not

CREATE TABLE t1

..

PRIMARY KEY

**((col1, col2),
col3, col4));**

```
CREATE MATERIALIZED VIEW t2_v1 AS
```

```
SELECT col1, ... , col8
```

```
FROM t2
```

```
WHERE
```

```
col1 IS NOT NULL AND col2 IS NOT NULL AND
```

```
col3 IS NOT NULL AND col6 IS NOT NULL
```

```
PRIMARY KEY (col1, col2, col3, col6);
```

```
// InvalidRequest: Error from server: code=2200 [Invalid query]
```

```
// message="Cannot create Materialized View t2_v1 without
```

```
// primary key columns from base t2 (col4)"
```



DSE Core: Secondary Indexes

```
SELECT * FROM T1  
WHERE col3 = 'ddd';
```



Scatter/gather

```
SELECT * FROM T1  
WHERE col3 = 'ddd'
```

```
AND col1 = 'ddd'  
AND col2 = 'ddd';
```



Node local

```
CREATE TABLE t1 ..  
PRIMARY KEY  
((col1, col2)) );
```

```
CREATE INDEX i2  
ON t1 (col3);
```

Secondary Indexes: You may not



```
SELECT * FROM T1 WHERE col3 > 'ddd';
```

```
// InvalidRequest: Error from server: code=2200 [Invalid query]  
// message="Cannot execute this query as it might involve data  
// filtering and thus may have unpredictable performance.  
// If you want to execute this query despite the performance  
// unpredictability, use ALLOW FILTERING"
```

Secondary Indexes: You may not



```
SELECT * FROM T1 WHERE col3 = 'ddd' ORDER BY col3;  
// InvalidRequest: Error from server: code=2200 [Invalid query]  
// message="ORDER BY with 2ndary indexes is not supported."
```

Secondary Indexes: You may not



```
CREATE INDEX i3 ON t1 (col5, col6);
```

```
// InvalidRequest: Error from server: code=2200 [Invalid query]
```

```
// message="Only CUSTOM indexes support multiple columns"
```

DSE Core: Two (?) Custom Indexes

An object titled, “custom index” displays in a DESCRIBE TABLE as the result of:

- **CREATE CUSTOM INDEX ...**

These are “SASI Indexes”, covered next-

- **CREATE SEARCH INDEX ...**

These are DSE Search indexes

```
CREATE TABLE ks_15.t1 (
```

```
col1 text, ...
```

```
DESCRIBE TABLE t1;
```

```
CREATE CUSTOM INDEX  
ks_15_t1_solr_query_index
```

```
ON ks_15.t1 (solr_query) USING  
'com.datastax.bdp.search.solr.Cql3SolrSecondaryIndex';
```

```
CREATE CUSTOM INDEX i5 ON ks_15.t1 (col7)  
USING
```

```
'org.apache.cassandra.index.sasi.SASIIndex';
```


DSE Core: (Custom) SASI Indexes



Beta

```
CREATE CUSTOM INDEX i5
ON ks_15.t1 (col7)
USING
'org.apache.cassandra.index.sasi.SASIIndex' ;

// Warnings :

// SASI index was enabled for 'ks_15.t1'. SASI is still
// in beta, take extra caution when using it in
// production.
```

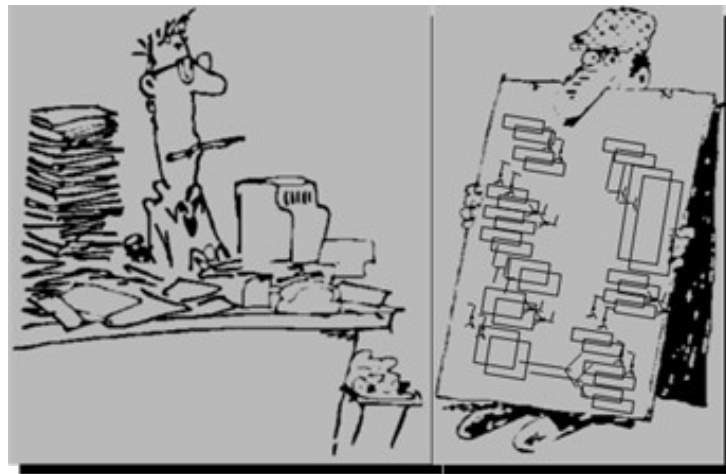
DSE Core: (Query Tables)

SQL/relational-

- Data modeled regardless of use
- Codd/Date, Third Normal Form
- No/few redundant attributes
- *Limiting cost factor, disk space/capacity*

Post-relational/NoSQL-

- Data modeled entirely by use
- **Derived/redundant data for performance**
- *Limiting cost factor, labor*



Practice Lab:

Design

Given 3 DSE Core CQL Queries, Create a Logical Data Model.

Practice Lab: DSE Core, Discuss How to Model-



- Select all guests that checked in at a particular hotel on a given day, sorted by time descending.
- Select all guest check ins for any given hotel within any given 10-minute window (E.g. 10:00-10:10, 13:42-13:52), regardless of day. Order the results by check in time descending.
- How many room-nights were at hotel 'Buena Vista' on 2015-12-15 ?

End of Practice Lab:

Discussion Lab:

Design

When to use Core, Search, (Analytics, Graph) ?

Discussion Lab: When to use Core, Search, (Analytics, Graph) ?

- SLA must be within 0-10 ms, or can be higher (100-250 ms+)
- Routine's frequency, concurrency, spread (capacity planning, cost)
- Syntactically/functionally, can it even be done



End of Discussion Lab:

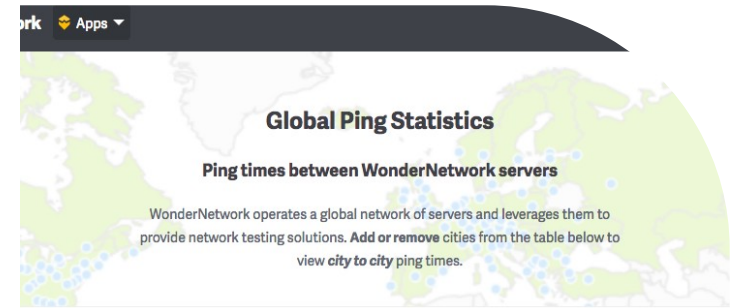
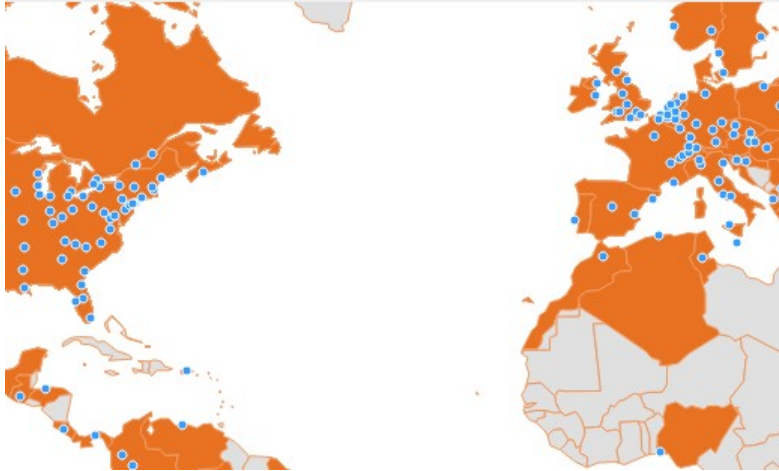


End of Module:



Additional Detail:

WAN Ping Latencies:



City	Barcelona	✖	Paris	✖	Tokyo	✖	Toronto	✖	Was
✖	36.888ms		29.754ms		263.752ms		92.956ms		9
✖	295.413ms		282.663ms		191.161ms		200.289ms		2
✖	64.438ms		25.831ms		243.51ms		105.168ms		9
✖	44.156ms		31.095ms		257.815ms		110.479ms		1
✖	147.664ms		142.856ms		134.454ms		43.565ms		3
✖	32.908ms		19.634ms		234.935ms		83.68ms		7
✖	154.71ms		141.967ms		107.634ms		58.913ms		6
✖	79.107ms		73.002ms		294.4ms		135.546ms		1
✖	107.253ms		115.799ms		206.435ms		20.112ms		8
✖	59.362ms		—		262.571ms		102.754ms		1
✖	64.363ms		59.331ms		288.42ms		126.434ms		1
✖	307.978ms		256.835ms		—		172.752ms		1

Source: <https://wondernetwork.com/pings>

000-DTSE-Search-7442-60-DM-35

Wikipedia.com: Tf-Idf Style Indexes

- “Nowadays, Tf-Idf is one of the most popular term-weighting schemes; 83% of text-based recommender systems in the domain of digital libraries use tf-idf.
- “Term frequency - the simplest choice is to use the *raw count* of a term in a document.
- “Inverse document frequency - is a measure of how much information the word provides, that is, whether the term is common or rare across all documents. It is the inverse fraction of the documents that contain the word, obtained by dividing the total number of documents by the number of documents containing the term.

WIKIPEDIA
The Free Encyclopedia



Source: <https://en.wikipedia.org/wiki/Tf-idf>
https://en.wikipedia.org/wiki/Bitmap_index

Tf-Idf (bitmap index) example-

Document ID

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
as	0	1	1	0	0	0	0	1	0	0	1	0	0	0	1	0	1	1	0	0
graceful	1	0	0	0	1	0	0	0	0	0	0	1	0	1	0	0	0	0	1	1
ponies	1	1	0	0	0	1	0	1	0	0	0	0	0	0	0	1	0	0	0	0
the	1	1	1	1	1	0	1	1	1	1	1	0	0	1	1	1	1	1	1	1
trot	1	0	0	1	0	0	1	0	0	0	0	0	1	0	0	0	0	0	1	0
unicorn	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0

The y-axis is populated with terms (tokens) extracted from a given column value.

The x-axis is the primary key to a given row.

1 represents true, zero false.

Which rows contain the value, “trot” ?

Which rows contain the values “trot” and “ponies” ?

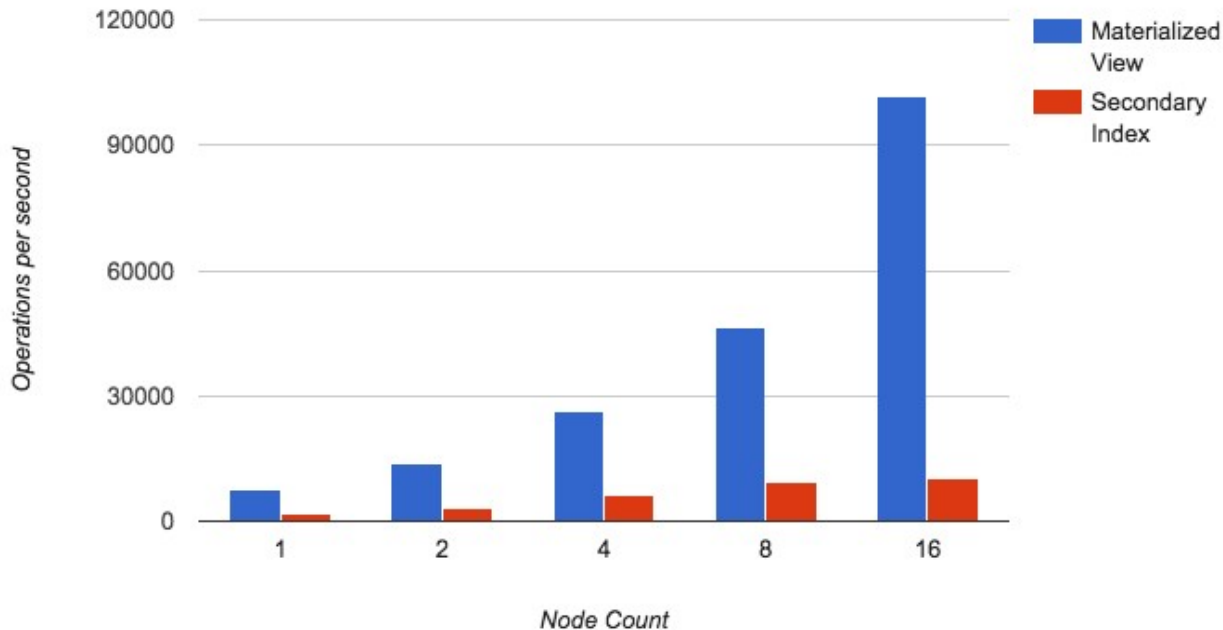
Product Compatibility Matrix: DSE and Cassandra

Determining which version of DSE maps
to which version of Cassandra-

Source:
https://docs.datastax.com/en/landing_page/doc/landing_page/compatibility.html

DataStax Enterprise	Cassandra
6.0	DSE 6.0 is compatible with Apache Cassandra™ 3.11 and adds additional production-certified changes
5.1	DSE production-certified 3.10 + enhancements
5.0	3.0 + enhancements
End of Life (EOL) — Patches/bug fixes not available.	
4.8	2.1 + enhancements
End of Service Live (EOSL) — Patches/bug fixes and support not available. Document	
4.7	2.1**
4.6	2.0
4.5	2.0
4.0	2.0
3.2	1.2
3.1	1.2
3.0	1.1
2.2	1.1
2.1	1.0
2.0	1.0
1.0	1.0

Performance: Mat.View versus Secondary Indexes



“The performance difference is dramatic even for small clusters, but even more important we see that indexed performance levels off when doubling from 8 to 16 nodes in the (AWS m3.xl) cluster, as the scatter/gather overhead starts to become significant.

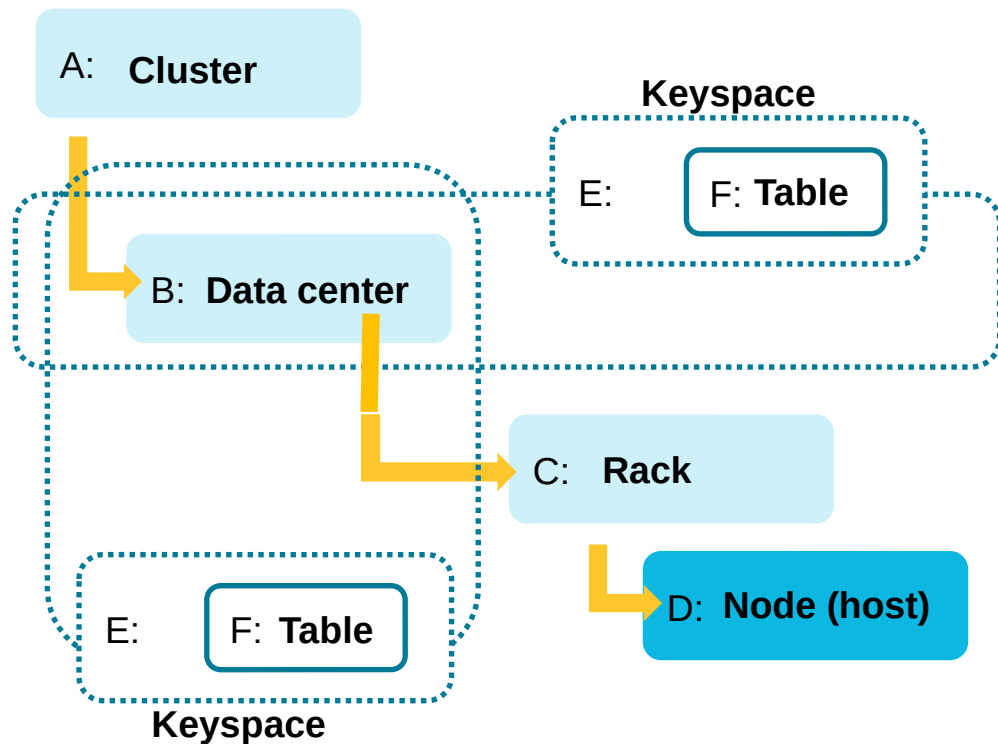
Source:

<https://www.datastax.com/dev/blog/materialized-view-performance-in-cassandra-3-x>



Solutions:

Discussion Lab: (One) DSE Object Hierarchy



$$E == G + H$$

**Keyspace == Replication Strategy
+ Replication Factor**

$$J == K + M$$

**Primary Key == Partitioning Key
+ Clustering Column**